

[Área personal](#) / [Mis cursos](#) / [Programación III](#) / [SEGUNDO PARCIAL - 1 de Junio de 2023](#) / [SEGUNDO PARCIAL JUEVES 01_06_2023](#)

Comenzado el	jueves, 1 de junio de 2023, 11:32
Estado	Finalizado
Finalizado en	jueves, 1 de junio de 2023, 12:46
Tiempo empleado	1 hora 13 minutos
Puntos	16,00/26,00
Calificación	6,15 de 10,00 (61,54%)

Pregunta **1**

Correcta

Se puntúa 1,00 sobre 1,00

La responsabilidad de gestionar la relación entre el observer y el observable debe ser de:

Seleccione una:

- ☐ a. El objeto observado que se extiende de **Observable**
- ☒ b. El objeto "observador" que implementa **Observer** ✓

Respuesta correcta

La respuesta correcta es: El objeto "observador" que implementa **Observer**

Pregunta **2**

Correcta

Se puntúa 1,00 sobre 1,00

Dada la clase genérica Caja<T>, siendo T el identificador del tipo de datos parametrizado, determine el valor de verdad de la variable rta:

```
Caja<Juguete> cajaJuguete = new Caja<Juguete>();  
Caja<Herramienta> cajaH = new Caja<Herramienta>();  
boolean rta = cajaJuguete.getClass() != cajaHerramienta().getClass();
```

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

Todas la invocaciones de clases genéricas son expresiones de una clase.

Al instanciar una clase genérica no se crea una nueva clase.

La respuesta correcta es 'Falso'

Pregunta **3**

Correcta

Se puntúa 1,00 sobre 1,00

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.
Según la clasificación, el patrón State es:

Correcta vale 1

Incorrecta vale -0,5

Seleccione una:

- ☒ a. de Comportamiento ✓
- ☐ b. Estructural
- ☐ c. Creacional

Respuesta correcta

de Comportamiento

La respuesta correcta es: de Comportamiento

Pregunta **4**

Correcta

Se puntúa 1,00 sobre 1,00

Para crear un hilo de ejecución y aplicar concurrencia se debe invocar desde la clase cliente al método **run()** de una clase que se se extienda de Thread

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta **5**

Correcta

Se puntúa 1,00 sobre 1,00

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.
El patrón State permite:

Correcta vale 1

Incorrecta vale -0,5

Seleccione una:

- ☐ a. Determinar un algoritmo general en la clase base y determinar los comportamientos particulares a través del mecanismo de herencia
- ☐ b. Permite añadir responsabilidades a un objeto en tiempo de ejecución
- ☒ c. Permitir que un objeto cambie el comportamiento al cambiar el estado ✓

Respuesta correcta

Permitir que un objeto cambie el comportamiento al cambiar el estado

La respuesta correcta es: Permitir que un objeto cambie el comportamiento al cambiar el estado

Pregunta **6**

Correcta

Se puntúa 1,00 sobre 1,00

En el patrón MVC, la Vista es la encargada de gestionar los eventos producidos por los componentes activos (botones, enlaces, etc).

Correcta: 1

Incorrecta: -0,5

- ☐ a. V
- ☒ b. F ✓

Respuesta correcta

La respuesta correcta es:

F

Pregunta **7**

Correcta

Se puntúa 1,00 sobre 1,00

De acuerdo al siguiente enunciado elija las opciones correctas.

Tenga en cuenta que existen dos clases principales que plantean dos formas diferentes de utilizar los métodos de ambos objetos

Valor respuesta correcta: 1

Valor respuesta incorrecta: -0,5

```
public class MiObserver implements Observer
{
    private MiObservable unObservable;
    //otros atributos

    public MiObserver()
    {
        //inicialización de otros atributos
    }

    public MiObserver(MiObservable unObservable)
    {
        this.setUnObservable(unObservable);
        this.getUnObservable().addObserver(this);
        //inicialización de otros atributos
    }

    @Override
    public void update(Observable o, Object arg)
    {
        // TODO Implement this method
    }

    public MiObservable getUnObservable()
    {
        return unObservable;
    }

    public void setUnObservable(MiObservable unObservable)
    {
        this.unObservable = unObservable;
    }
}
```

```
public class MiObservable extends Observable
{
    //atributos especificos

    public MiObservable()
    {
        super();
    }
}
```

```
public class Main (opcion 1)
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver(modelo);
    }
}
```

```
public class Main (opcion 2)
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver();
        modelo.addObserver(ojo);
    }
}
```

Seleccione una:

- ☐ a. Opcion 1 y 2 funcionan, sin embargo la opcion 2 es la que respeta el patrón, debido a un tema de asignación de responsabilidades y prevención de errores
- ☐ b. el código de la clase Main en la opcion 2 no funciona
- ☐ c. Opcion 1 y 2 son igualmente válidas.
- ☒ d. Opcion 1 y 2 funcionan, sin embargo la opcion 1 es la que respeta el patrón, debido a un tema de asignación de responsabilidades y prevención de errores 

Respuesta correcta

La respuesta correcta es: Opcion 1 y 2 funcionan, sin embargo la opcion 1 es la que respeta el patrón, debido a un tema de asignación de responsabilidades y prevención de errores

Pregunta **8**

Correcta

Se puntúa 1,00 sobre 1,00

La función específica de la instrucción wait() es evitar que dos o más hilos accedan en forma simultánea a un recurso compartido

Correcta: 1
Incorrecta: -1

- ☐ a. V
- ☒ b. F 

Respuesta correcta

La respuesta correcta es:

F

Pregunta **9**

Correcta

Se puntúa 1,00 sobre 1,00

En el patrón MVC, es conveniente que la clase vista cree el modelo y el controlador para poder dar inicio al programa.

Correcta: 1

Incorrecta: -0,5

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

La respuesta correcta es:

F

Pregunta **10**

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos el caso de un objeto de tipo Observable que está siendo observado por varios objetos de tipo Observer a la vez. En ese contexto, el patrón Observer Observable está pensado para:

Correcta: 1

Incorrecta: -1

Seleccione una:

- ☒ a. Que un objeto observable notifique a todos los objetos observer a la vez. ✓
- ☐ b. que un objeto observable notifique a un objeto observer a elección.

Respuesta correcta

El patrón permite que un objeto observable notifique a todos sus observers a la vez

La respuesta correcta es: Que un objeto observable notifique a todos los objetos observer a la vez.

Pregunta **11**

Sin contestar

Puntúa como 5,00

De un ejemplo de un caso donde sea conveniente utilizar el patrón State.

El ejemplo no debe ser ninguno de los vistos en clase ni en el TPE.

Bosqueje la solución haciendo un diagrama de clases completo (o escribiendo de las clases la jerarquía de herencia, los atributos y las cabeceras de métodos).

Establezca los atributos que determinarán los diferentes estados del objeto, determine los estados, los métodos, etc.

Solamente el diagrama de clases con atributos y cabecera de métodos.

No es necesario que haga un gráfico.

Pregunta **12**

Correcta

Se puntúa 1,00 sobre 1,00

En el Patrón MVC.

¿De quién es la responsabilidad de validar el formato correcto de los datos que el usuario ingresa?

Seleccione una:

- ☐ a. Controlador
- ☒ b. Vista ✓
- ☐ c. Modelo

Respuesta correcta

La respuesta correcta es: Vista

Pregunta **13**

Correcta

Se puntúa 1,00 sobre 1,00

Los objetos referenciados por las variables v1 y v2 son de distinto tipo

```
Sea una Clase genérica Clase1<T> y sean:  
Clase1<String> v1 = new Clase1<String>();  
Clase1<Integer>v2 = new Clase2<Integer>();
```

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

Las respuestas correctas son:

F,

V

Pregunta **14**

Correcta

Se puntúa 1,00 sobre 1,00

Determinar si es de buen diseño la definición de la clase XXX.

```
public class XXX extends Thread
{
    private int dato;
    public synchronized XXX ( int dato )
    {
        this.dato=dato;
    }
    //resto del código
}
```

Correcta: 1

Incorrecta: -1

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

No es un buen diseño.

Los constructores no se pueden sincronizar, no tiene sentido.

La clase ejemplificada es un Thread, no un recurso compartido.

La respuesta correcta es:

F

Pregunta **15**

Correcta

Se puntúa 1,00 sobre 1,00

Para poder persistir un objeto utilizando Serealización Binaria

Seleccione una:

- ☐ a. Tener un constructor público sin parámetros
- ☐ b. Ninguna de las anteriores
- ☒ c. Todos los elementos que queremos persistir deben implementar la interfaz Serializable ✓
- ☐ d. Tener un constructor público sin parámetros y getters y setters públicos

Respuesta correcta

La respuesta correcta es: Todos los elementos que queremos persistir deben implementar la interfaz Serializable

Pregunta **16**

Correcta

Se puntúa 1,00 sobre 1,00

Para poder persistir un objeto utilizando archivos XML es necesario

Seleccione una:

- ☐ a. Ninguna de las anteriores
- ☐ b. Tener un constructor que nos permita pasar por prámetro los atributos que queremos persistir
- ☒ c. Tener un constructor público sin parámetros y getters y setters públicos ✓
- ☐ d. Que todos sus métodos sean públicos y sus atributos privados

Respuesta correcta

La respuesta correcta es: Tener un constructor público sin parámetros y getters y setters públicos

Pregunta **17**

Sin contestar

Puntúa como 5,00

Explique la diferencia entre synchronized y wait().

Para qué sirve cada uno. Explique también para qué se usa la *condición* y por qué va dentro de un while.

```
public synchronized void metodo()
{
    while( //condición... )
    {
        wait();
    }
}
```

Establezca un ejemplo (un problema), relátelo muy brevemente y determine para qué se utilizaría synchronized y para qué se utilizaría wait().

Pregunta **18**

Correcta

Se puntúa 1,00 sobre 1,00

Un método estático no puede ser sincronizado ya que el método start() de Thread es de instancia y no estático.

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

La respuesta correcta es:

F

[◀ Ejemplo Persistencia cuando hay patron Singleton](#)

Ir a...